Unit 6: Optimization

Chapter 14 in "Statistical Computing with R"

Anna Ly

Department of Mathematical and Computational Sciences
University of Toronto Mississauga

March 16, 2026

Overview

1. Introduction
2. Optimize() and Optim()
3. Newton-Raphson Method
4. EM Algorithm

Introduction

Introduction

- You've seen optimization methods before: the MLE.
- We'll discuss the following (non-exhaustive list):
    - R's standard `optimize()` function.
    - R's `optim()` function, and commonly used methods (Nelder-Mead, BFGS, L-BFGS-B, SANN, Brent).
    - Newton Raphson.
    - EM Algorithm.

Optimize() and Optim()

Optimize()

- According to the documentation, the method used is a combination of golden section search and successive parabolic interpolation to find when the **maximum** occurs.
- Unfortunately, the source code for `optimize()` is actually written in C! See here! (Click link). Same goes for `optim()`.
- In the interest of time, this discussion focuses on how to use `optimize()` and `optim()` rather than their underlying implementation.
- This function efficiently locates the maximum of one-dimensional functions.

Optimize()

Example

Use `optimize()` to maximise this function with respect to x:

$$f(x) = \frac{\log(1 + \log(x))}{\log(1 + x)}$$

Solution. Consult the R code.

Optimize()

Example

Recall this practice question: consider the Pareto distribution:

$$f_X(x) = \frac{\alpha\beta^\alpha}{x^{\alpha+1}}, \quad x > \beta.$$

Let $X_1, X_2, \ldots, X_n$ be a random sample from a Pareto distribution where $\beta = 2$, and we want to estimate α. Derive the MLE for α. Then, use `optimize()` to do it for us.

Solution.

$$L(\alpha \mid \mathbf{X}) = \alpha^n 2^{2n} \prod_{i=1}^{n} x_i^{-(\alpha+1)}$$

$$l(\alpha \mid \mathbf{X}) = n\ln(\alpha) + \alpha n \ln(2) - (\alpha+1)\sum_{i=1}^{n} \ln(x_i)$$

Optimize()

$$\frac{d\, l(\alpha \mid \mathbf{X})}{d\alpha} = \frac{n}{\alpha} + n\ln(2) - \sum_{i=1}^{n} \ln(x_i)$$

$$0 \overset{set}{=} \frac{n}{\alpha} + n\ln(2) - \sum_{i=1}^{n} \ln(x_i)$$

$$\hat{\alpha}_{MLE} = \frac{n}{\sum_{i=1}^{n} \ln(x_i) - n\ln(2)}$$

Optim()

`optim()` is more popular than `optimize()` because you can use this for multi-dimensional problems and can accommodate a wide variety of methods:

1. Nelder-Mead
2. BFGS (Broyden-Fletcher-Goldfarb-Shanno)
3. CG (Conjugate gradient)
4. L-BFGS-B (L = Limited-memory, B = Bound-constrained)
5. SANN (Simulated-Annealing)
6. Brent; one-dimension only, same as `optimize()`.

These methods are graduate-level topics, but they are readily accessible through `optim()`. Note that by default, `optim()` minimises; to obtain the maximum, negate the objective function. The following example compares their performance across methods.

Optim()

Example

1. Derive the likelihood function $L(\hat{\boldsymbol{\theta}}) = L(\hat{\alpha}, \hat{\lambda})$ for $X_1, \ldots, X_n \sim Gamma(\alpha, \lambda)$ where α is the shape parameter and λ is the rate parameter.
2. Then, for $n = 10^4$ randomly sample from a $Gamma(\alpha = 5, \lambda = 2)$ using `rgamma()`.
3. Use `optim()` to maximize this function with respect to $\boldsymbol{\theta} = (\alpha, \beta)$ and compare the results between the 5 different methods.

Solution. Let $\boldsymbol{x} = (x_1, \ldots, x_n)$ be the realizations of $X_1, \ldots, X_n$:

$$
\begin{aligned}
L(\boldsymbol{\theta}|\mathbf{x}) &= \prod_{i=1}^{n} \frac{\lambda^{\alpha}}{\Gamma(\alpha)\beta^{\alpha}} x_i^{\alpha-1} e^{-\lambda x_i} \\
&= \frac{\lambda^{n\alpha}}{\Gamma(\alpha)^n} \left(\prod_{i=1}^{n} x_i \right)^{\alpha-1} e^{-\lambda \sum_{i=1}^{n} x_i}
\end{aligned}
$$

Optim()

$$\ln(L(\boldsymbol{\theta}|\mathbf{x})) = -\ln(\Gamma(\alpha)) + n\alpha\ln(\lambda) + (\alpha - 1)\sum_{i=1}^{n}\ln(x_i) - \lambda\sum_{i=1}^{n}x_i$$

See the corresponding R codes.

Example

1. Derive the likelihood function $L(\hat{\boldsymbol{\theta}}) = L(\hat{\mu}, \hat{\sigma})$ for $X_1, \ldots, X_n \sim Normal(\mu, \sigma)$.
2. Then, for $n = 10^4$ randomly sample from a $Normal(\mu = 2, \sigma = 4)$ using `rnorm()`.
3. Use `optim()` to maximize this function with respect to $\boldsymbol{\theta} = (\alpha, \beta)$ and compare the results between the 5 different methods.

Solution. Let $\boldsymbol{x} = (x_1, \ldots, x_n)$ be the realizations of $X_1, \ldots, X_n$:

$$L(\boldsymbol{\theta}|\mathbf{x}) = (2\pi\sigma^2)^{-n/2} \exp\left\{\frac{-1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2\right\}$$

$$l(\boldsymbol{\theta}|\mathbf{x}) = \frac{-n}{2}\ln(2\pi\sigma^2) + \frac{-1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

See the corresponding R codes.

Optim()

Example

Use `optim()` to maximise this function with respect to x:

$$f(x, n, r, t) = \left(1 + \frac{n-r}{2}\right) x - r + \left(\frac{n-r}{2}\right) t$$

Compare the results between the 5 different methods.

Solution. Consult the R code.

Newton-Raphson Method

Newton-Raphson Method

- The Newton-Raphson method, named after Isaac Newton and Joseph Raphson, is a root-finding algorithm that produces successively better approximations to the roots of a real-valued function.
- This method is particularly useful when it is not possible to derive a closed-form expression for the likelihood function, and consequently, the MLE.
- We will use Newton-Raphson to numerically maximise the log-likelihood.
- The method is introduced first in the one-variable case before extending to multiple variables.

Understanding Newton-Raphson - One Variable Case

- Suppose we test one value for our function, call it x_0.
- Draw a tangent line to $f(x)$ at x_0. This tangent line will intersect the x-axis at some fixed point $(x_1, 0)$.
- Draw another tangent line to $f(x)$ at x_1; the tangent line will intersect the x-axis at some point $(x_2, 0)$...
- Repeat the above steps until you find a value of x_i such that $f(x_i) = 0$.

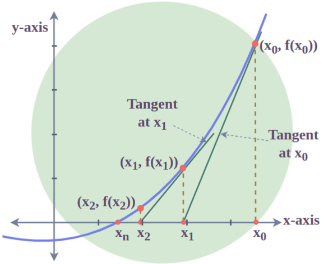

Image source: https://www.geeksforgeeks.org/engineering-mathematics/newton-raphson-method/

Understanding Newton-Raphson - One Variable Case

- Recall from calculus: the Taylor polynomial of order n generated by $f(x)$ at $x = a$ is the polynomial

$$P_n(x) = f(a) + f'(a)(x - a) + \frac{f^{(2)}(a)}{2}(x - a)^2 \cdots + \frac{f^{(n)}(a)}{n!}(x - a)^n$$

- The linear approximation around x_0 is:

$$P_1(x) = f(x_0) + f'(x_0)(x - x_0)$$

We want $P_1(x_0) = 0$:

$$0 = f(x_0) + f'(x_0)(x - x_0) \;\Rightarrow\; x = x_0 - \frac{f(x_0)}{f'(x_0)}$$

Understanding Newton-Raphson - One Variable Case

Newton-Raphson - One Variable

In the general form, the Newton-Raphson method formula is written as follows:

$$x_n = x_{n-1} - \frac{f(x_n)}{f'(x_n)}$$

We now want to extend this to multiple variables.

Understanding Newton-Raphson - Multiple Variable Case

- Remember the **Jacobian**: "the Jacobian matrix of a vector-valued function of several variables is the matrix of all its first-order partial derivatives."
- Let $\mathbf{x} = (x_1, x_2, \ldots, x_k)$ and $\mathbf{x_0} = (x_{01}, x_{02}, \ldots, x_{0k})$
- Thus instead we can do:

$$f(\mathbf{x}) \approx f(\mathbf{x_0}) + J(\mathbf{x_0})(\mathbf{x} - \mathbf{x_0})$$

- We can solve for $\mathbf{x}$ the same way:

$$\mathbf{x} = \mathbf{x_0} - J(\mathbf{x_0})^{-1} f(\mathbf{x_0})$$

- Assuming the Jacobian is invertible.

Modifying Newton-Raphson for our Case

- Consider $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_k)$ and some values $\boldsymbol{\theta}^* = (\theta_1^*, \theta_2^*, \ldots, \theta_k^*)$.
- Note that when we are solving for the MLE, we are not finding the roots. What we need to do is maximise the likelihood function $L(\boldsymbol{\theta})$.
- We know the likelihood is maximized (or minimized) when the derivative of the likelihood $\mathcal{L}(\boldsymbol{\theta}^*) = \frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}\Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} \overset{set}{=} 0$.
- And now we need to consider the Hessian matrix (call $H(\cdot)$), which includes the second-order partial derivatives.
- Thus, borrowing the equation from the previous slide but adjusting it to our new situation, we now have the following:

$$\hat{\boldsymbol{\theta}} = \boldsymbol{\theta}^* - H(\boldsymbol{\theta}^*)\mathcal{L}(\boldsymbol{\theta}^*)$$

Newton-Raphson Method

Newton-Raphson for Solving the MLE

1. Choose a reasonable starting value $\boldsymbol{\theta}^{(0)}$.
2. Update your estimate:

$$\boldsymbol{\theta}^{(n)} = \boldsymbol{\theta}^{(n-1)} - H(\boldsymbol{\theta}^{(n-1)})\mathcal{L}(\boldsymbol{\theta}^{(n-1)})$$

3. Terminate the algorithm when $||\boldsymbol{\theta}^{(n)} - \boldsymbol{\theta}^{(n-1)}|| < \epsilon$ where ϵ is a small threshold. In this scenario, we're letting $|| \cdot ||$ represent the Euclidean norm:

$$||\mathbf{x}|| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

Newton-Raphson Method

Example

Consider the same log likelihood function we solved earlier for the Gamma distribution. Again, randomly sample values from *Gamma*$(\alpha = 5, \lambda = 2)$ and now use the Newton-Raphson Method to solve for the MLE. *Hint:* you're allowed to use `numDeriv::grad` and `numDeriv::hessian`.

Solution. Consult the R code.

Newton-Raphson Method

Example

Consider the same log likelihood function we solved earlier for the Normal distribution. Again, randomly sample values from *Normal*$(\mu = 2, \sigma = 4)$ and now use the Newton-Raphson Method to solve for the MLE. *Hint:* you're allowed to use `numDeriv::grad` and `numDeriv::hessian`.

Solution. Consult the R code.

Expectation-Maximization Algorithm

EM Algorithm

- Missing data arises in virtually every applied field, including machine learning, survival analysis, and the social sciences.
- The goal is still to construct good estimators despite incomplete observations.
- Rather than deriving a closed-form expression for the MLE directly, we specify an algorithm that is guaranteed to converge to the MLE.
- It is based on the idea of replacing one difficult likelihood maximisation with a sequence of easier maximisations whose limit is the answer to the original problem.

EM Algorithm

1. Let $\mathbf{Y}$ be the complete data with log likelihood $l_c(\boldsymbol{\theta}; \mathbf{Y})$.
2. Let $\mathbf{X}$ be the observed (incomplete) data with log likelihood $l(\boldsymbol{\theta}; \mathbf{X})$.
3. Choose an initial estimate $\hat{\boldsymbol{\theta}}^{(0)}$.
4. **E-step:** Calculate:
$$\mathbf{Q}(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}^{(k)}) = \mathbb{E}[l_c(\boldsymbol{\theta}; \mathbf{Y}) | \hat{\boldsymbol{\theta}}^{(k)}, \mathbf{X}]$$
5. **M-step:** Choose $\hat{\boldsymbol{\theta}}^{(k+1)}$ to maximise $\mathbf{Q}(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}^{(k)})$.
6. Iterate the **E** and **M** step until:
$$\mathbf{L}(\hat{\boldsymbol{\theta}}^{(k+1)}) - \mathbf{L}(\hat{\boldsymbol{\theta}}^{(k)}) < \epsilon$$

EM Algorithm

- Consider iid $X_1, \dots, X_n$ where $X_i \sim Poisson(\tau)$ for $i = 1, 2, \dots, n$. Suppose we had all of the realizations $x_2, x_3, \dots, x_n$ except for X_1.
- **Idea:** discard X_1 as well and then just compute and maximise:

$$L(\tau | X_2 = x_2, X_3 = x_3, \dots, X_n = x_n)$$

- **Any issues with the idea?**

EM Algorithm

Example

It is still useful to compute the incomplete likelihood:

$$L(\tau | X_2 = x_2, X_3 = x_3, \ldots, X_n = x_n)$$

So derive this as well as the MLE to get an initial estimate.

Solution.

$$L(\tau|\text{Data}) = \prod_{i=2}^{n} f(x_i) = \left[\prod_{i=2}^{n} \frac{e^{-\tau}(\tau)^{x_i}}{x_i!}\right] = e^{-(n-1)\tau}\tau^{\sum_{i=2}^{n} x_i} \prod_{i=2}^{n} \frac{1}{x_i!}$$

Since $\prod_{i=2}^{n} \frac{1}{x_i!}$ is free of τ, the derivative of this constant w.r.t τ will be 0, so we can ignore it when deriving the MLE. Then...

EM Algorithm

$$l(\tau|\text{Data}) = \ln\left(e^{-(n-1)\tau}\tau^{\sum_{i=2}^{n} x_i}\right) = -(n-1)\tau + \sum_{i=2}^{n} x_i \ln(\tau)$$

$$\frac{dl(\tau|\text{Data})}{d\tau} = -(n-1) + \frac{1}{\tau}\sum_{i=2}^{n} x_i$$

$$0 \overset{\text{set}}{=} -(n-1) + \frac{1}{\tau}\sum_{i=2}^{n} x_i$$

$$\hat{\tau}_{MLE} = \frac{\sum_{i=2}^{n} x_i}{n-1}$$

We will use this as a reference for our initial estimate $\hat{\tau}^{(0)}$.

EM Algorithm

Example

For the same example as before, compute the expectation step and then compute the maximisation step:

$$\mathbb{E}\left[l(\tau|\mathbf{x}) \mid \hat{\tau}^{(k)}, \mathbf{x}_{-1}\right]$$

Solution. Again, we compute the likelihood, but now this is complete:

$$L(\tau|\mathbf{x}) = \prod_{i=1}^{n} f(x_i) = \left[\prod_{i=1}^{n} \frac{e^{-\tau}(\tau)^{x_i}}{x_i!}\right] e^{-n\tau} \tau^{\sum_{i=1}^{n} x_i} \prod_{i=1}^{n} \frac{1}{x_i!}$$

Similarly, $\prod_{i=1}^{n} \frac{1}{x_i!}$ does not depend on τ and can be ignored when differentiating.

EM Algorithm

Thus,

$$\ln(L(\tau|\mathbf{x})) = -n\tau + x_1 \ln(\tau) + \sum_{i=2}^{n} x_i \ln(\tau)$$

Let's compute the expected value:

$$\mathbb{E}\left[l(\tau|\mathbf{x}) \mid \hat{\tau}^{(k)}, \mathbf{x}_{-1}\right] = \mathbb{E}\left[-n\tau + x_1 \ln(\tau) + \sum_{i=2}^{n} x_i \ln(\tau) \mid \hat{\tau}^{(k)}, \mathbf{x}_{-1}\right]$$

$$= -n\tau + \ln(\tau)\mathbb{E}[X_i|\hat{\tau}^{(k)}, \mathbf{x}_{-1}] + \ln(\tau)\sum_{i=2}^{n} x_i$$

$$\mathbf{Q}(\tau, \hat{\tau}^{(k)}) = -n\tau + \ln(\tau)\hat{\tau}^{k} + \ln(\tau)\sum_{i=2}^{n} x_i$$

EM Algorithm

Now we can take the derivative (maximisation step):

$$
\begin{aligned}
\frac{\partial \mathbf{Q}(\tau, \hat{\tau}^{(k)})}{\partial \tau} &= -n + \frac{\hat{\tau}^k}{\tau} + \frac{1}{\tau}\sum_{i=2}^{n} x_i \\
0 &\overset{set}{=} -n + \frac{\hat{\tau}^k}{\tau} + \frac{1}{\tau}\sum_{i=2}^{n} x_i \\
n\tau &= \hat{\tau}^k + \sum_{i=2}^{n} x_i \\
\hat{\tau}^{(k+1)} &= \frac{\hat{\tau}^k + \sum_{i=2}^{n} x_i}{n}
\end{aligned}
$$

For $k = 1, 2, \ldots$

EM Algorithm

Example

Explicitly code up the EM algorithm for the case we just considered.

Solution. Consult the R code.

EM Algorithm

Example

Consider $X_i \overset{i.i.d}{\sim} Normal(\mu, \sigma^2)$ where $i = 1, 2, \dots, n$ where $n = 10^4$. Suppose we had all of the realisations except for one of them, let's say X_1. Suppose σ^2 is known, but not the mean μ. Construct the EM algorithm and code it in R.

Solution. We begin by finding $\hat{\mu}^{(0)}$ by completing the incomplete log likelihood. Note that,

$$L(\mu|\mathbf{x}_{-1}) = (2\pi\sigma^2)^{-(n-1)/2} \exp\left\{\frac{-1}{2\sigma^2}\sum_{i=2}^{n}(x_i - \mu)^2\right\}$$

$$l(\mu|\mathbf{x}_{-1}) = \frac{-(n-1)}{2}\ln(2\pi\sigma^2) - \left\{\frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i - \mu)^2\right\}$$

Next, we need to take the derivative.

$$\frac{d\, l(\mu|\mathbf{x}_{-1})}{d\mu} = -\frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i - \mu)(2)(-1)$$

$$0 \overset{set}{=} \frac{1}{\sigma^2}\sum_{i=2}^{n} x_i - (n-1)\mu$$

$$\hat{\mu}^{(k)} := \frac{\sum_{i=2}^{n} x_i}{n-1}$$

Note that the complete log-likelihood is:

$$l_c(\mu|\mathbf{x}) = \frac{-n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

$$= \frac{-n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}(x_1 - \mu)^2 - \frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i - \mu)^2$$

EM Algorithm

Expectation step:

$$
\begin{aligned}
&\mathbb{E}[l_c(\mu|\mathbf{x}) \mid \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2] \\
&= \mathbb{E}\left[\frac{-n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}(x_1-\mu)^2 - \frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i-\mu)^2 \,\middle|\, \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2\right] \\
&= \frac{-n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\mathbb{E}\left[(x_1-\mu)^2 \,\middle|\, \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2\right] - \frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i-\mu)^2
\end{aligned}
$$

Note that:

$$
\begin{aligned}
\mathbb{E}\left[(x_1-\mu)^2 \,\middle|\, \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2\right] &= \mathbb{E}\left[x_1^2 - 2\mu x_1 + \mu^2 \,\middle|\, \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2\right] \\
&= \mathbb{E}\left[x_1^2 \,\middle|\, \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2\right] - 2\mu\mathbb{E}\left[x_1 \,\middle|\, \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2\right] + \mu^2 \\
&= \sigma^2 + (\hat{\mu}^{(k)})^2 - 2\mu\hat{\mu}^{(k)} + \mu^2
\end{aligned}
$$

Thus,

$$\mathbb{E}[l_c(\mu|\mathbf{x}) \mid \hat{\mu}^{(k)}, \mathbf{x}_{-1}, \sigma^2]$$
$$= \frac{-n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\left[\sigma^2 + (\hat{\mu}^{(k)})^2 - 2\mu\hat{\mu}^{(k)} + \mu^2\right] - \frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i - \mu)^2$$

And so we begin the maximisation step:

$$Q(\mu, \hat{\mu}^{(k)}) := \frac{-n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\left[\sigma^2 + (\hat{\mu}^{(k)})^2 - 2\mu\hat{\mu}^{(k)} + \mu^2\right] - \frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i - \mu)^2$$

$$\frac{\partial Q(\mu, \hat{\mu}^{(k)})}{\partial \mu} = \frac{-1}{2\sigma^2}(-2\hat{\mu}^{(k)}) + \frac{-1}{2\sigma^2}(2\mu) - \frac{1}{2\sigma^2}\sum_{i=2}^{n}(x_i - \mu)(2)(-1)$$
$$= \frac{\hat{\mu}^{(k)}}{\sigma^2} - \frac{\mu}{\sigma^2} + \frac{1}{\sigma^2}\sum_{i=2}^{n}(x_i - \mu)$$

EM Algorithm

Setting equal to 0 to find the maximum of μ (verification that this is a maximum via the second derivative is left as an exercise):

$$0 = \frac{\hat{\mu}^{(k)}}{\sigma^2} - \frac{\mu}{\sigma^2} + \frac{1}{\sigma^2}\sum_{i=2}^{n}(x_i - \mu)$$

$$0 = \hat{\mu}^{(k)} - \mu + \sum_{i=2}^{n} x_i - (n-1)\mu$$

$$\mu = \frac{\hat{\mu}^{(k)} + \sum_{i=2}^{n} x_i}{n}$$

And thus we set:

$$\hat{\mu}^{(k+1)} := \frac{\hat{\mu}^{(k)} + \sum_{i=2}^{n} x_i}{n}$$

Consult the corresponding R codes.

EM Algorithm

Some remarks regarding the EM algorithm:

- Typically, the maximization step, i.e., maximizing $\mathbb{E}[l_c(\boldsymbol{\theta}; \mathbf{Y})|\hat{\boldsymbol{\theta}}^{(k)}, \mathbf{X}]$ is VERY difficult and cannot done manually. Thus, we tend to rely on numerical methods, such as Newton-Raphson, to do this!
- In practice, the maximisation step often cannot be completed in closed form. For the purposes of examinations, questions will either:
  - Ask you to derive parts UP until the expectation step. No maximisation explicitly required; I may ask you "what could you do to maximise this if you had the tools?"
  - Give you a case that is possible to derive, similar to practice problems and lecture slides.

EM Algorithm

Let's consider the second case – that is, we will not be able to find closed-form expressions, but we can still compute these things numerically.

Example

Consider $X_i \overset{i.i.d}{\sim} Gamma(\alpha, \lambda)$ where $i = 1, 2, \ldots, n$ where $n = 100$. Suppose we had all of the realisations except for one of them, let's say X_1. Suppose both α (the shape parameter) and λ (the rate parameter) are unknown. Construct the EM algorithm and code it in `R`.

(Exercise for you!) Note that the complete log-likelihood is:

$$l_c(\alpha, \lambda \,|\, \mathbf{x}) = \alpha n \ln(\lambda) - n \ln(\Gamma(\alpha)) + (\alpha - 1) \sum_{i=1}^{n} \ln(x_i) - \lambda \sum_{i=1}^{n} x_i,$$

EM Algorithm

The incomplete log likelihood is then:

$$l(\alpha, \lambda \,|\, \mathbf{x}_{-1}) = \alpha(n-1)\ln(\lambda) - (n-1)\ln(\Gamma(\alpha)) + (\alpha - 1)\sum_{i=2}^{n} \ln(x_i) - \lambda \sum_{i=2}^{n} x_i.$$

Note that we will use `optim()` to find a suitable starting point for $\hat{\alpha}^{(0)}, \hat{\lambda}^{(0)}$. (You could also use Newton-Raphson if you prefer.) Now, for the expectation step:

$$\begin{aligned}
&\mathbb{E}[l_c(\alpha, \lambda \,|\, \mathbf{x}) \,|\, \hat{\alpha}^{(k)}, \hat{\lambda}^{(k)}, \mathbf{x}_{-1}] \\
&= \alpha n \ln(\lambda) - n\ln(\Gamma(\alpha)) + (\alpha - 1)\sum_{i=2}^{n} \ln(x_i) - \lambda \sum_{i=2}^{n} x_i \\
&+ \mathbb{E}[(\alpha - 1)\ln(x_1) \,|\, \hat{\alpha}^{(k)}, \hat{\lambda}^{(k)}, \mathbf{x}_{-1}] + \mathbb{E}[\lambda x_1 \,|\, \hat{\alpha}^{(k)}, \hat{\lambda}^{(k)}, \mathbf{x}_{-1}]
\end{aligned}$$

EM Algorithm

Note that,

$$\mathbb{E}[\lambda x_1 \mid \hat{\alpha}^{(k)}, \hat{\lambda}^{(k)}, \mathbf{x}_{-1}] = \lambda \frac{\hat{\alpha}^{(k)}}{\hat{\lambda}^{(k)}}$$

For $Y \sim Gamma(\alpha, \lambda)$ there is a "nice" closed form for $\mathbb{E}[\log(Y)]$. However, this involves a weird calculus trick and I'm LAZY.

Instead, Monte Carlo methods (Unit 2) can be used to approximate this quantity.

Generate $Y_j \sim Gamma(\hat{\alpha}^{(k)}, \hat{\lambda}^{(k)})$ where $j = 1, \ldots, m$ and m represents our MC sample size. Then, using SLLN let:

$$\mathbb{E}[\log(X_1)] \approx \frac{1}{m} \sum_{j=1}^{m} \log(Y_j)$$

EM Algorithm

Hence, now we want to optimize to find the value of α, λ of the following expression:

$$Q(\alpha, \lambda, \hat{\alpha}^{(k)}, \hat{\lambda}^{(k)}) = \alpha n \ln(\lambda) - n \ln(\Gamma(\alpha)) + (\alpha - 1) \sum_{i=2}^{n} \ln(x_i) - \lambda \sum_{i=2}^{n} x_i$$
$$+ (\alpha - 1) \frac{1}{m} \sum_{j=1}^{m} \log(y_j) + \lambda \frac{\hat{\alpha}^{(k)}}{\hat{\lambda}^{(k)}}$$

Similar to how we found the initial values for $\hat{\alpha}^{(0)}, \hat{\lambda}^{(0)}$, we can use `optim()` to do the maximization step for us due to a lack of having a closed form.

Check the remaining `R` codes.